

Cold-Chain Logistics : AI-Assistant

Prerequisites :

Core Programming & Data Fundamentals: Python, OOP, **SQL**-Database

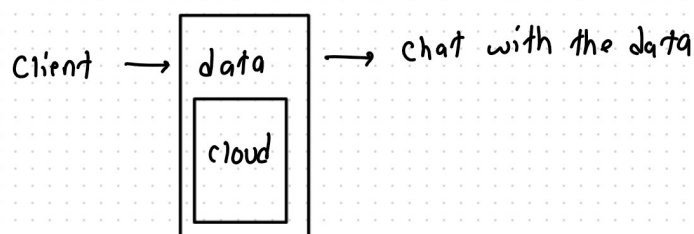
Artificial Intelligence & LLM Core: LLM, Vector-Database

Advanced AI Agents & Frameworks: Langgraph, AI-Agent, Tools

Development Environment & Tools: VSCode, Conda/Venv

User Interface & Deployment: Streamlit, , Docker Desktop, AWS

problem statement



legacy system → MS-SQL 2022 → api X

→ id
→ password }

→ dashboard ← api ← database

UI → dropdown
→ learning curve
→ charts
→ SOP

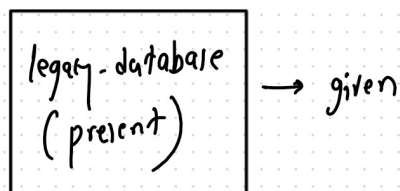
2024 - Jan - 01 - 15th

→ analysis
→ 100 answers/metrics

→ agentic-ai / chatbot
→ answer

→ database → legacy system → EC2 (running)

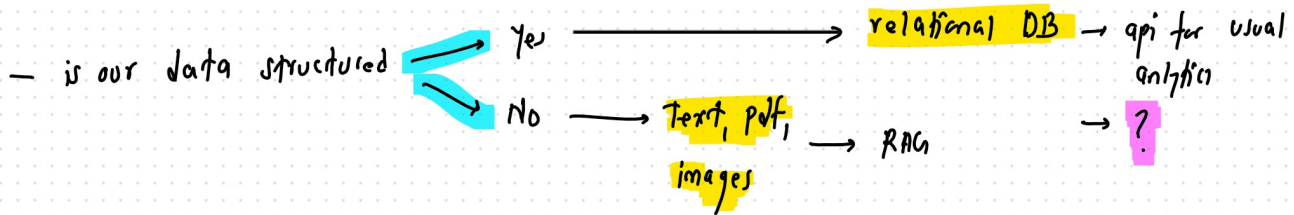
→ problem statement (known) → solution discussion → presentation



SOP → given

Solution Discussion

what do we mean by "talk to our data"



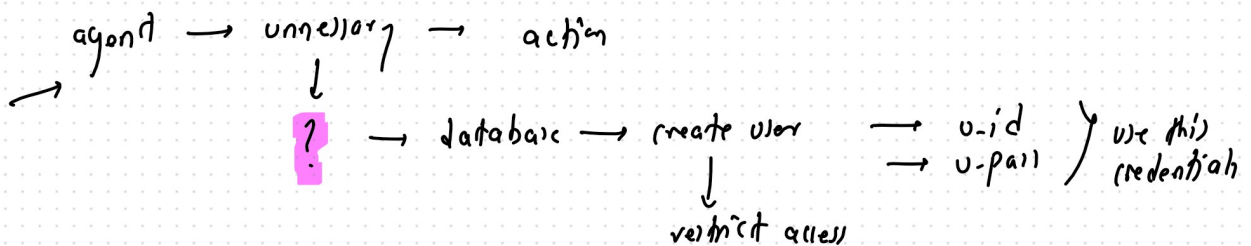
Query details

- based on location
- based on coordinate
- perishable
- temperate/veg
- weather condition
- combination of above

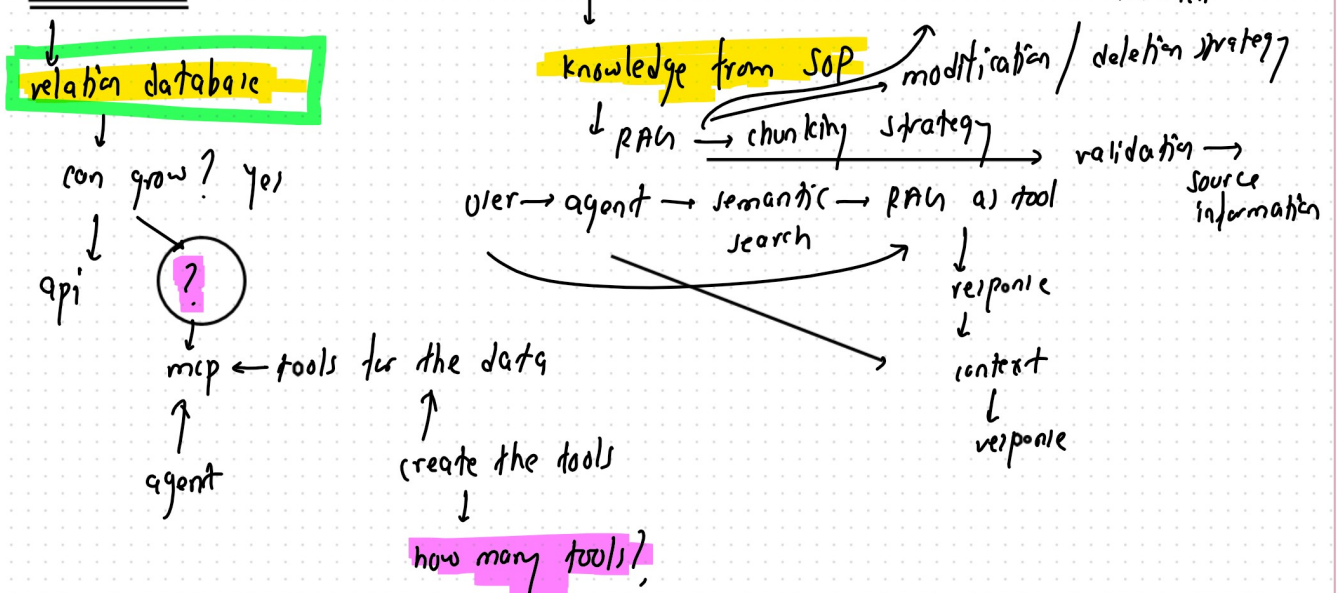
→ query from sop!

Security → agent does not insert/delete the data.

what does it take to build this with security in mind?



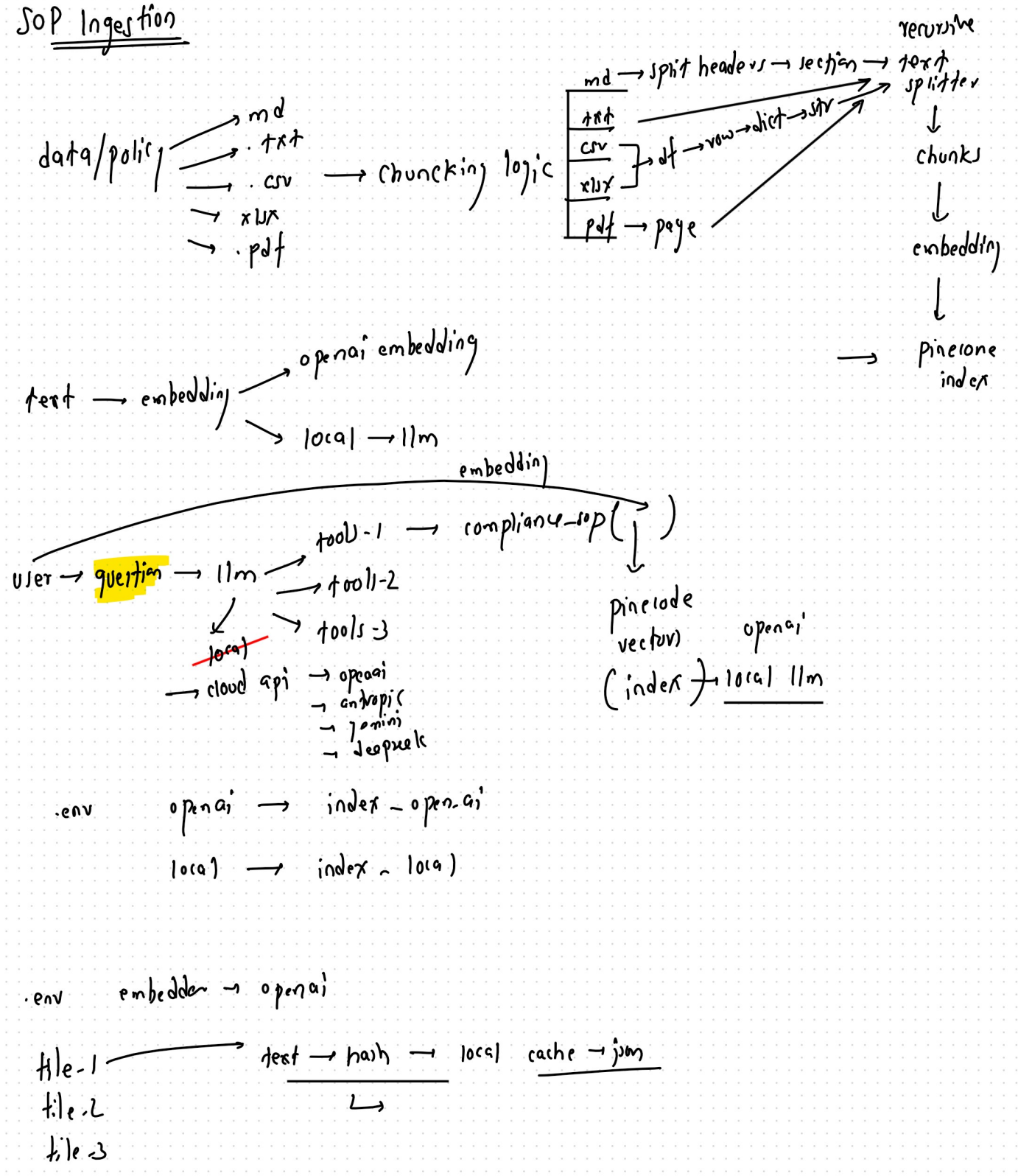
data source

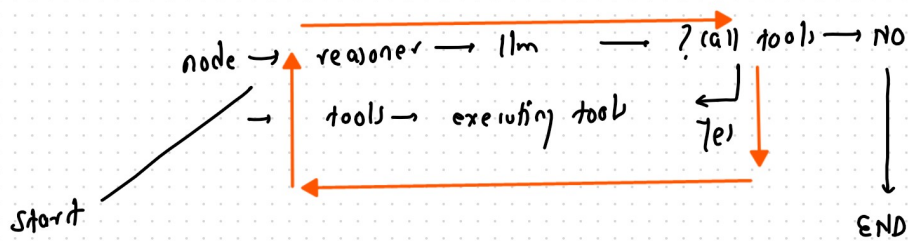


← **text2SQL**

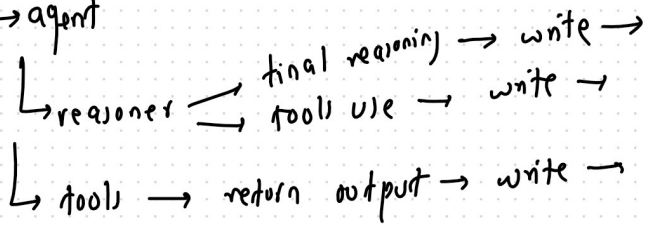
- ← cons: query is not correct, we will not get result → agent that can interact & see the previous response
- ← pro: not limited → multistep
- : faster prototype

SOP Ingestion



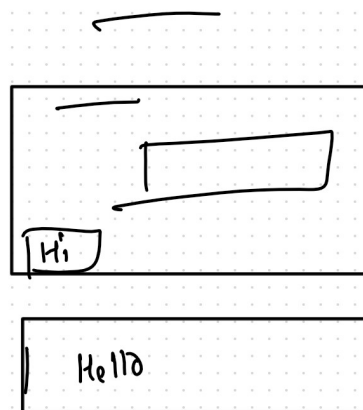


input → agent

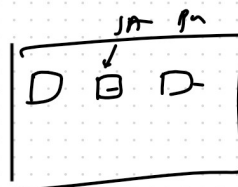


FDE_views. AgentAuditlog
 INVERT ↑
 WR-FDE-RO;

streamlit → interact → rerun
 ↓
 loading the model → cache
 st.
 st.session



Security → Chatbot →



Introduction → prerequisite → problem statement → Deep dive into the data

↓
Designed the solution

↓
System
Architecture
(DOC)

← The Business
pitch ←

Development

- step
- 1: Building injection logic
 - 2: configured database roles and permission
 - 3: Developed core tools for Agent
 - 4: orchestrator the agent
 - 5: Build the streamlit ui
 - 6: Automated Deployed via Github actions to AWS EC2